

AIOps Module 1: Experiment Management & Reproducibility

Mohammed Khaja Ghouse Mohiuddin (DA24B015)

Partner: Mudda Manikanta Pruthvi Raj

1. Technical Debt Diagnosis

(a) Entanglement (CACE): Modifying the delivery-time feature's rounding logic unexpectedly degraded the unrelated favorite-restaurants feature. In ML models, the weight matrix couples all input features together, so changing one feature's preprocessing shifts behavior on unrelated outputs.

(b) Undeclared Consumers: The marketing team was silently reading the model's raw output table without the ML team knowing. This creates an invisible dependency. Any change to the output schema or score distribution by the ML team would break marketing's pipeline without warning.

(c) Glue-Code Debt (Pipeline Jungle): 14 undocumented shell scripts with no orchestration. No parameterization, no reproducibility, and tribal knowledge is needed to run it.

Mitigation for (c): Replace the scripts with an MLflow Project. An `MLproject` file defines named entry points with declared parameters and types, and a `python_env.yaml` pins dependencies. The pipeline becomes runnable via `mlflow run . -P lr=0.01`, and each run is auto-tracked in MLflow.

2. MLflow Experiment Comparison

Six experiments trained a two-layer MLP (`MLPClassifier`) on MNIST, varying learning rate and hidden layer sizes. All runs used `batch_size=200`, `seed=42`, and 20 epochs.

Run	Learning Rate	Hidden Layers	Batch Size	Accuracy
1	0.001	(256, 128)	200	0.9751
2	0.001	(128, 64)	200	0.9725
3	0.01	(128, 64)	200	0.9694
4	0.0001	(128, 64)	200	0.9627
5	0.01	(256, 128)	64	0.9608
6	0.001	(64, 32)	200	0.9603

Best run: `lr=0.001` with hidden layers (256, 128) at 97.51% accuracy. Larger network with a moderate learning rate gave the best test performance.

Overfitting: Train loss dropped from 0.39 to 0.01 over 20 epochs, but validation accuracy flattened around epoch 5 at about 96.6% and barely moved after that. The model kept fitting the training data without improving on unseen data.

Hyperparameter effect: Learning rate had a bigger impact than architecture. Holding hidden layers at (128, 64), changing `lr` from 0.001 to 0.0001 dropped accuracy by about 1%. Architecture changes across the same `lr` showed a similar range but the change was more gradual.

3. DVC Data Versioning and Rollback

DVC tracked a CSV of image filenames. `v1` had 1800 rows; after appending new entries, `v2` had 2801 rows. Both versions were pushed to an SSH remote. To roll back, I ran `git checkout v1` then `dvc checkout`, which restored the file to exactly 1800 rows (`wc -l`: 1801 with header). Going back to `master` and running `dvc checkout` again restored `v2` (`wc -l`: 2802). This shows DVC correctly restores data when the `.dvc` pointer file is reverted.

4. End-to-End Reproducibility

Partner A: Trained an MLP on DVC-tracked MNIST data. Logged all params (`lr`, hidden layers, epochs, batch size, seed), the final accuracy (0.9782), and a `git_commit` tag in MLflow. Code and `.dvc` file were committed together. The model was registered as `my-classifier` and moved to Staging.

Partner B: Cloned the repo, set up the env from `environment.yml`, pulled data via DVC SSH remote, and re-ran training. Got 0.9784 accuracy (diff of 0.0002 from Partner A's 0.9782, well within the 0.005 tolerance). The small difference comes from floating-point behavior on different hardware. A reproducibility note was logged in MLflow confirming the match.

The full protocol (Git + DVC + MLflow + pinned environment) worked as expected for independent reproduction.